

We have gone through the description/classification of hypervisors from different perspectives - software, hardware, full/para/hardware-assisted etc. Let us understand that these are not separate, competing options. Instead, they are different **layers** of classification. This means that a single hypervisor is being described using multiple terms at once.

Layer 1: Classification by Location (Where is the hypervisor seated in relation to the physical host h/w?)

Type-1 (Bare-Metal): Installs directly onto the physical hardware. There is no host operating system underneath it. VMware ESXi, KVM, Xen are some examples. These are good for Enterprise datacenters and cloud computing (high performance).

Query: If there is no host OS, how does this work? A bare-metal hypervisor includes core OS components **scaled** down strictly for virtualization:

Type-2 (Hosted): Installs as software inside a traditional host operating system. VirtualBox, VMware Workstation, Parallel Desktop are examples. These are good for local software development and testing on a personal laptop. They are rarely (or almost not) used in production datacentres as performance is limited. These also lack a lot of advanced features like live migration of VMs.

Layer 2: Classification by Architecture (How does the guest OS communicate with the host CPU & memory?)

Full Virtualization: The guest OS is not modified. It is completely unaware that it has been virtualized. The hypervisor intercepts hardware commands and emulates a fake environment.

Para-Virtualization: The guest OS is modified on purpose. It knows it is virtualized and uses special software API commands ("hypercalls") to communicate with the hypervisor for resources. This enables skipping the slow emulation phase.

Hardware-Assisted Virtualization: The hypervisor relies on physical features built directly into modern CPU microchips (examples: Intel VT-x or AMD-V). The silicon handles the heavy translation. This enables full virtualization to offer a great performance - native hardware speeds.

Note: **x86 hardware virtualization** (often called hardware-assisted virtualization) refers to the physical CPU modifications made by Intel and AMD to allow multiple operating systems to run simultaneously on x86 processors without performance loss.

Layer 3: Classification by Scope (What is being virtualized/What is the boundary of the virtual environment?)

Hardware Virtualization (Server/OS): Simulates a complete physical computer system (CPU, RAM, Storage). We install the guest OS of our choice. This includes everything listed above (Layer 1 and Layer 2)

Software/OS-Level Virtualization (Containers): The host OS (Kernel) is shared, but user spaces are isolated. There is no hypervisor. There is no guest OS. Docker, LXC are examples. Microservices and lightweight application deployment.

One common misnomer:

When we discuss VMware, VirtualBox, or Hyper-V in the context of "software vs. hardware," we need to understand/analyze them from the context of **how the hypervisor translates CPU instructions**, not the scope of what is virtualized.

Software-Based Virtualization (Binary Translation): The hypervisor uses pure software code to intercept, analyze, and rewrite the guest OS instructions on the fly before passing them to the physical CPU. This is how the older/legacy versions of VirtualBox and Early VMware (ESX) worked before Intel and AMD added virtualization features to their chips. This is slow because the software handles all translation.

Hardware-Assisted Virtualization: Modern versions of VirtualBox, VMware, and Hyper-V do not use software binary translation. They require physical CPU extensions (Intel VT-x/AMD-V) to run. The hardware chip does the heavy lifting, making the software hypervisor vastly faster.



Work Integrated Learning Programmes

Let us see an example, where in we cover the different aspects of classification.

AWS Nitro - offloads virtualization, network and storage tasks from the host CPU to dedicated hardware

Interested students can refer to this link - <https://aws.amazon.com/ec2/nitro/>

When AWS launches an ec2 instance (HVM - hardware virtual machine) via the AWS Nitro, it satisfies the following three categories - type 1, Full virtualization and hardware-assisted.

Location: Type-1 - Directly on the physical metal (bare metal)

The Guest OS is unmodified - Full Virtualization

Execution is Hardware-assisted (Intel VT-x/AMD-V)



Work Integrated Learning Programmes

Now that we have under virtualization, how do we see this from another perspective, viz, the three service models – IaaS, PaaS and SaaS.

Resource management in virtualization: This involves splitting physical hardware into smaller parts. These parts are assigned to different service models like IaaS, PaaS, and SaaS to keep performance stable and fair.

Infrastructure as a Service (IaaS)

- **Full control:** The user gets a full virtual machine, storage, and network.
- **Direct allocation:** The system sets strict limits for CPU cores, RAM size, and disk space.
- **Tools used:** Hypervisors like KVM or VMware ESXi handle the raw hardware sharing.

Platform as a Service (PaaS)

- **Managed runtime:** User only manages code and data.
- **System manages resources by Auto-scaling:** The system adds or removes resources on demand based on traffic load.
- **Tools for Container sharing:** Tools like Kubernetes share the same OS kernel while keeping apps separate.

Software as a Service (SaaS)

- **No infrastructure control:** User has no control over the infra. The provider runs everything from hardware to application logic.
- **Multi-tenant pooling:** Many users share the same application pool and database resources safely. Note: DBs may be shared/allocate separately for each tenant
- **Fair sharing:** The system uses load balancers to stop one user from using all the system power.